

SourceWise Study Agent

Phase 1 — Frontend Development Guide

This document provides a complete, step-by-step specification for building the SourceWise frontend using React 18, Tailwind CSS, and Vite. It covers every screen, component, state management pattern, API integration strategy, and tech stack decision required to deliver a polished, production-ready web application before any backend work begins.

SourceWise Study Agent • Project Documentation • 2025

1. Tech Stack & Project Setup

The frontend is a single-page application (SPA) bootstrapped with Vite for near-instant hot-module replacement. All UI components are styled with Tailwind CSS utility classes for consistency and speed. Global client-side state is managed with Zustand — a lightweight, hook-based alternative to Redux. Axios is configured as a centralized HTTP client with interceptors for JWT injection and error handling.

Library / Tool	Purpose & Notes
Vite 5 + React 18	Build tool & UI framework. Fast HMR, JSX support, tree-shaking.
Tailwind CSS 3	Utility-first CSS. All spacing, colors, and typography via classes.
React Router v6	Client-side routing with nested layouts and protected routes.
Zustand	Minimal global state store for auth, sources, planner, quiz progress.
Axios	HTTP client. Single instance with baseURL, request/response interceptors.
React Query (TanStack)	Server-state caching, background refetching, loading/error states.
Recharts	Radar chart (topic coverage), bar chart (daily study hours), ring chart.
React Dropzone	Drag-and-drop file upload zone with MIME type validation.
React Hook Form	Form handling for auth, exam date picker, topic selection.
Framer Motion	Page transitions, card flip animation for flashcards, modal spring.
shadcn/ui	Pre-built accessible components: Dialog, Toast, Badge, Tabs, Slider.
Wavesurfer.js	Audio waveform player for the revision audio module.

Project scaffold command:

```
npm create vite@latest sourcewise-frontend -- --template react
npm install tailwindcss @tailwindcss/forms zustand axios @tanstack/react-query recharts
react-dropzone react-hook-form framer-motion
```

2. Folder Structure

```
src/ assets/ # Logos, icons, illustrations components/ # Reusable UI atoms (Button, Card, Badge, Modal) features/ # Feature-level modules auth/ # Login, Signup, ForgotPassword upload/ # SourceUpload, FileList, ParseProgress summary/ # TopicCard, SummaryPanel, ConceptExplainer flashcards/ # FlashCard, CardDeck, StudySession quiz/ # QuizQuestion, OptionButton, ScoreBoard audio/ # AudioPlayer, TranscriptPanel, SpeedControl planner/ # WeekCalendar, DayPlan, MissedAlert dashboard/ # ReadinessRing, TopicRadar, WeakTopicList hooks/ # useAuth, useSources, usePlanner, useQuiz lib/ # axiosInstance, queryClient, constants pages/ # Route-level page components store/ # Zustand slices (authSlice, sourceSlice, planSlice) router/ # AppRouter, ProtectedRoute, PublicRoute utils/ # dateUtils, scoreCalculator, fileValidator App.jsx main.jsx
```

3. Pages & Screen Specifications

Step 3.1 — Landing / Auth Pages

- Full-screen split layout: left branding panel (gradient), right form panel.
- Login form: email + password, 'Remember me' checkbox, 'Forgot Password' link.
- Signup form: name, email, password, confirm password. Client-side validation via React Hook Form.
- JWT stored in httpOnly cookie (preferred) or localStorage with refresh token rotation.
- Redirect to /dashboard after successful auth. Protected route wrapper blocks unauthenticated access.
- Social auth placeholder buttons (Google) — disabled until OAuth is wired in Phase 2.

Step 3.2 — Source Upload Module

- Full-width drag-and-drop zone using React Dropzone. Accepts PDF, DOCX, TXT, PPTX.
- Each uploaded file shows: filename, type badge (color-coded), file size, upload progress bar.
- After upload, backend returns parsed chunks — show 'Processing...' spinner with estimated time.
- Uploaded sources listed in sidebar with checkboxes to select which sources are active for queries.
- Delete button per source with confirmation modal. Source count badge in nav.
- Error state: unsupported format shows toast notification with allowed types.

Step 3.3 — Summary & Concept View

- Left panel: source tree (accordion) showing document → chapters → topics.
- Right panel: selected topic's AI-generated summary rendered as formatted text with headings.
- Difficulty badge per topic: Easy (green), Medium (amber), Hard (red).
- Source citation chips below each paragraph linking back to the relevant uploaded document.
- Regenerate button to request a fresh AI summary for the selected topic.
- Copy-to-clipboard button on each summary block.

Step 3.4 — Flashcard Module

- Card deck view: 3D flip animation (Framer Motion rotateY) on click. Front = question, back = answer.
- Progress bar at top showing cards reviewed / total cards.

- Action buttons: Know it (green check), Review Again (red X), Skip (gray arrow).
- Keyboard shortcuts: Space to flip, Right arrow = Know it, Left arrow = Review Again.
- Filter bar: filter cards by topic, difficulty, or source document.
- Generate More button calls API to produce additional cards from the same source material.

Step 3.5 — Quiz Module

- MCQ layout: question at top, 4 option buttons below. Single correct answer per question.
- Timer bar (optional) counts down per question — configurable in settings.
- After each answer: highlight correct (green) and selected wrong (red), show explanation.
- Score board at end: percentage, time taken, topic-wise breakdown, weak areas highlighted.
- Retry button (full quiz or only wrong answers), Export results as PDF button.
- Difficulty selector (Easy / Mixed / Hard) before quiz starts.

Step 3.6 — Audio Revision Player

- Wavesurfer.js waveform visualizer with play/pause, seek scrubber, and time display.
- Playback speed selector: 0.75x, 1x, 1.25x, 1.5x, 2x.
- Topic selector dropdown to load audio for any uploaded source topic.
- Collapsible transcript panel below player — scrolls in sync with audio (word-level if TTS supports it).
- Download MP3 button. Share as link button (generates a short-lived signed URL).
- Mini player persists at the bottom of all pages when audio is playing (like Spotify).

Step 3.7 — Study Planner

- Exam configuration form: subject name, exam date, daily study hours available.
- Generated timetable displayed as a weekly grid — each cell is a topic + estimated time.
- Day detail view: click any day to expand into a prioritized topic list with time blocks.
- Missed session alert: if today's plan is incomplete, banner shows 'Catch Up' button.
- Catch-up mode redistributes missed topics across remaining days automatically.
- Color coding: completed sessions (green), in-progress (amber), pending (gray), overdue (red).

Step 3.8 — Dashboard & Readiness Score

- Hero card: large readiness percentage ring (Recharts RadialBarChart) with color thresholds.
- Topic coverage radar chart: each axis = a topic, filled area shows how much has been studied.
- Weak topics list: ranked by lowest quiz scores, click to jump directly to that topic's flashcards.
- Study streak widget: consecutive days studied, best streak, total hours.
- Recent activity timeline: last 5 actions (quiz completed, flashcard session, audio played).
- Exam countdown: days remaining badge per registered exam.

4. State Management Architecture

Zustand is used for client-owned state (auth tokens, UI preferences, active source selection). React Query handles all server-state — it caches API responses, refetches in the background, and provides isLoading / isError flags to every component without prop drilling.

Zustand Slice	State & Actions
authSlice	user object, accessToken, isAuthenticated, login(), logout()
sourceSlice	uploadedSources[], activeSourceIds[], uploadProgress
planSlice	examDates[], weeklyPlan, missedSessions[], catchUpMode
quizSlice	currentQuestion, selectedAnswer, score, sessionHistory[]
uiSlice	sidebarOpen, activeTheme, miniPlayerVisible, toastQueue[]

5. API Layer & Mock Data Strategy

During Phase 1, all API calls return mock data from local JSON files. The Axios instance is configured once with the base URL as an environment variable — switching from mock to real backend requires only changing VITE_API_BASE_URL in .env.

```
// src/lib/axiosInstance.js
import axios from 'axios'; const api = axios.create({ baseUrl:
import.meta.env.VITE_API_BASE_URL }); api.interceptors.request.use(cfg => { const token
= useAuthStore.getState().accessToken; if (token) cfg.headers.Authorization = `Bearer
${token}`; return cfg; }); export default api;
```

Endpoint	Response Shape
POST /auth/login	Returns { user, accessToken, refreshToken }
POST /sources/upload	Multipart form. Returns { sourceId, name, status }
GET /sources/:id/summary	Returns { topics: [{ name, summary, difficulty }] }
GET /flashcards?sourceId=	Returns { cards: [{ id, question, answer, topic }] }
POST /quiz/generate	Body: { sourceId, count, difficulty }. Returns quiz object.
GET /audio/:topicId	Returns { audioUrl, transcript }
POST /planner/generate	Body: { examDates, hoursPerDay }. Returns weeklyPlan.
GET /dashboard/readiness	Returns { score, topicCoverage, weakTopics, streak }

6. Routing Structure

```
/ → Landing / Login /signup → Registration page /dashboard → Main dashboard
(protected) /sources → Source upload & management (protected) /summary/:sourceId →
Topic summary viewer (protected) /flashcards → Flashcard study session (protected)
/quiz → Quiz module (protected) /audio → Audio revision player (protected) /planner →
Study timetable (protected) /settings → User preferences (protected) /* → 404 not found
page
```

7. Frontend Deployment

Platform: Vercel (recommended) — auto-deploys on every Git push to main.

Build command: npm run build (outputs to /dist)

Environment vars: VITE_API_BASE_URL=https://api.sourcewise.app

CI/CD: GitHub Actions: lint → test → build → deploy on PR merge.

Custom domain: sourcewise.app via Vercel domain settings. Auto SSL/TLS.

Performance: Vite code-splits routes automatically. Lazy load heavy pages.

End of Phase 1 — Frontend Document | SourceWise Study Agent | 2025